

A **Language Model** computes the probability distribution over sequences of tokens/subtokens. It assigns a probability P to plausible sequences (e.g., $P(\text{I have read}) > P(\text{eye have red})$).

Applications: Machine translation, spell check, speech recognition, and standalone systems like ChatGPT (QA, summarization).

Markov Assumption Instead of conditioning on the *entire* history, we approximate by considering only the **current state**.

First-order Markov Assumption: The future is conditionally independent of the past given the present: $P(S_{t+1}|S_0, \dots, S_t) \approx P(S_{t+1}|S_t)$ A **Homogeneous Markov Chain** assumes transition probabilities are constant across all time steps t .

1 **Discrete Markov Chains**

State Space (S): Finite set of unique observations $\{s_1, s_2, \dots, s_n\}$.
Initial Distribution (π_0): Probability distribution over states at $t = 0$.

Transition Matrix (T): Matrix A where $a_{ij} = P(s_j|s_i)$ is the prob of moving from s_i to s_j .

Transition matrix for a MM means calculating the proportion of how often sequences of states occur in your data. Two corpora would have different word n-gram frequencies.

Note: Each row must sum to 1.0.

1.1 **Key Computations**

1. **Sequence Probability:** Using the Markov assumption, a sequence prob is the product of conditionals: $P(s_0, s_1, s_2) = P(s_0) \times P(s_1|s_0) \times P(s_2|s_1)$ 2. **State Distribution at time t (π_t):** Can be calculated using matrix multiplication: $\pi_t = \pi_{t-1}T$ or $\pi_t = \pi_0T^t$

```
pi_0 = np.array([0.5, 0.3, 0.2]) # initial state probability dist
T = np.array([[0.5, 0.2, 0.3], [0.2, 0.5, 0.3], [0.3, 0.1, 0.6]])
pi_18 = pi_0 @ np.linalg.matrix_power(T, 18) # State at t=18
```

2 **Stationary Distributions**

Steady State Analysis A distribution π is **stationary** if it remains unchanged as time progresses. $\pi T = \pi$ It describes the long-run behavior of the chain (e.g., PageRank algorithm).

2.1 **Conditions for Uniqueness**

A unique stationary distribution exists if the chain is:

- **Irreducible:** Possible to reach any state from any other state (fully connected graph).
- **Aperiodic:** No fixed repetitive cycles. GCD of return times to any state is 1. Self-loops help break periodicity.

2.2 **How to find Stationary distributions?**

Power Iteration: Multiply π_0 by T repeatedly until stable. **Eigen-decomposition:** Transpose T and find the eigenvector with eigenvalue $\lambda = 1$. **Monte Carlo:** Simulate many transitions and count occurrences.

If a state has only one possible transition, the transition probability for that transition would be 1.0. If each row in the transition matrix of a Markov chain has only one possible transition, the chain would be deterministic. **Fully Connected:** Irreducible (Yes) — Aperiodic (Yes) **Fixed Cycles:** Irreducible (Yes) — Aperiodic (No) **Absorbing State:** Irreducible (No) — Aperiodic (Yes)

3 **Markov Models (MM)**

Markov Assumption: Future is conditionally independent of the past given the present.

Learning = Counting: Estimates use Maximum Likelihood Estimates (MLE).

Initial Dist (π_0): $P(s_i) = \frac{\# \text{ starts in } s_i}{n \text{ sequences}}$

Transitions (T): $P(s_j|s_i) = \frac{\# \text{ moves } s_i \rightarrow s_j}{\# \text{ moves } s_i \rightarrow \text{anything}}$

Matrix Property: Sum of each row in T must be 1.0.

```
#Step 1 Tokenize
toy_corpus_tokens = nltk.word_tokenize(toy_corpus.lower())
#Step 2: Count transitions
frequencies = defaultdict(Counter)
for i in range(len(toy_corpus_tokens) - 1):
    frequencies[toy_corpus_tokens[i] : i + 1][0][toy_corpus_tokens[i + 1]] += 1
#Step 3: Convert to probabilities (Transition matrix)
trans_df = freq_df.div(freq_df.sum(axis=1), axis=0)
```

4 **N-gram Language Models: MM of Language**

With n-gram models, we are able to incorporate more history. calculate probability of sequences, predict the state at a given time step, or calculate the stationary distribution the same way.

4.1 **Limitations**

Sparsity: State space grows exponentially as V^{n-1} (V =vocab size).
Long-range Dependencies: Cannot capture context beyond the fixed n window.

No Semantics: Treats words as discrete symbols (dog≠cat).

5 **Sampling Techniques:Text Generation**

Greedy: Always picks highest probability word; can be repetitive.

Top-k: Keeps top k words, renormalizes, and samples;adds diversity.

Top-p (Nucleus): Samples from smallest set whose cumulative probability exceeds p .

Temperature (τ): Reshapes distribution via $y = \text{softmax}(u/\tau)$.

Low τ (< 1): More deterministic, more like greedy sampling.

High τ (> 1): more diverse, useful for creative and open-ended tasks.

6 **Evaluation of Language models**

Increasing the value of n in MM usually results in lower perplexity.

Intrinsic: Perplexity: Inverse probability of test set normalized by N words. $\sqrt[N]{1/P(w_1 \dots w_N)}$.

Lower Perplexity better predictor of the words in the test set.

Higher Perplexity n typically decreases perplexity.

Evaluating capabilities of modern LLMs, it's not sufficient. The difference in perplexity between high-performing models may not correlate with actual quality of output (e.g., helpfulness, relevance, factuality). **Extrinsic:** The most reliable way to evaluate a llm is to embed it into a downstream application and measure how much the performance of the application improves.(translation, reasoning). Eg of some benchmarks - Reading comprehension tasks, World knowledge tests, Common sense reasoning.

7 **Hidden Markov Models (HMM)**

The 5-Tuple $\{S, Y, \pi, A, B\}$ Used when states are unobserved (latent). **S:** Hidden states (e.g., Moods). **Y:** Observations (e.g., Activities). **A:** Transition matrix $P(q_i|q_{i-1})$. **B:** Emission probabilities $P(o_i|q_i)$.

HMM Assumptions— Markov: State depends only on previous state. **O/P Independence:** Obs depends only on current hidden state.

8 **HMM Fundamentals**

An HMM is defined by $\theta = \langle \pi, A, B \rangle$:

π : Initial state distribution $P(q_1)$. A : Transition matrix $a_{ij} = P(q_t|q_{t-1})$. B : Emission probabilities $b_j(o_t) = P(o_t|q_t)$.

Independence Assumptions: o_t is conditionally independent of past obs and states given q_t . q_t is conditionally independent of past states/obs given q_{t-1} .

9 **Task 1: Likelihood (Forward Algorithm)**

Goal: Compute $P(O|\theta)$ for sequence $O = o_1, \dots, o_T$.

Naive approach: $P(O) = \sum_Q P(O, Q)$ is intractable (n^T paths).

Forward Variable $\alpha_i(t)$: $P(o_1 \dots o_t, q_t = s_i|\theta)$, the probability of seeing partial obs and ending in state s_i .

9.1 **Forward Steps — Dynamic Programming**

Initialize: $\alpha_i(1) = \pi_i b_i(o_1)$. **Induction:** $\alpha_j(t) = \sum_{i=1}^n \alpha_i(t-1) a_{ij} b_j(o_t)$. **Termination:** $P(O|\theta) = \sum_{i=1}^n \alpha_i(T)$.

Complexity: $O(n^2T)$ via Dynamic Programming.

Observation sequence O and the model parameters are fixed and known.

Total probability of seeing the first t observations and ending up in state i at time t , considers all possible paths that could have led there. $\alpha_i(t)$ does not know anything about the future time steps after the time step.

Forward procedure by summing over the values at the last Time step. You can pass sequences of different lengths when training HMMs.

10 **Task 2: Decoding (Viterbi Algorithm)**

The Viterbi algorithm is a dynamic programming approach used to find the **most likely sequence of hidden states** (the Viterbi path) given a sequence of observations.

1. **Initialization ($t = 1$)** At the very first time step, we calculate the starting probability for each state. "What is the probability that we started in state i and saw the first observation o_1 ?"

2. **Induction ($2 \leq t \leq T$)** For every subsequent time step, we move forward through the sequence. $\delta_j(t) = \max_{1 \leq i \leq N} [\delta_i(t-1) a_{ij}] b_j(o_t)$ $\psi_j(t) = \arg \max_{1 \leq i \leq N} [\delta_i(t-1) a_{ij}]$

• $\delta_j(t)$ **(The Score):** This finds the "best" path to reach state j

at time t . We look at all possible previous states i , pick the one that was most likely to transition to j , and multiply by the current observation probability.

• $\psi_j(t)$ **(The Breadcrumb):** This records *which* previous state i was the winner. We don't need the probability here, just the "ID" of the best previous state so we can retrace our steps later.

3. **Termination** Once we reach the end of the sequence (T), we identify the single most likely ending state. "Of all the possible final states, which one has the highest total path probability?"

4. **Backtracking ($t = T - 1, T - 2, \dots, 1$)** $q_t^* = \psi_{q_{t+1}^*}^*(t + 1)$ We

follow the breadcrumbs (ψ) from the end of the sequence back to the beginning. "Since I know I ended in state X , I check my notes (ψ) to see which state most likely led me to X . I repeat this until I have the full path."

11 **Task 3: Supervised vs Unsupervised Learning**

Supervised: Use MLE (counting) if state sequences are known.

Unsupervised: Use iterative **Expectation-Maximization** (Baum-Welch) if only O is given.

12 **Recurrent Neural Networks (RNN)**

12.1 **Foundations & Motivation**

RNNs are supervised models designed for **sequential data**. Unlike feedforward networks, they use **hidden units** to retain information over time. Hidden states are **continuous, high-dimensional**, and much richer than discrete states in HMMs.

12.2 **Weight Matrices & Parameters**

RNNs utilize three core weight matrices:

• W : Input to hidden state ($x_t \rightarrow h_t$).

• U : Hidden to hidden ($h_{t-1} \rightarrow h_t$).

• V : Hidden to output ($h_t \rightarrow y_t$).

Parameter Sharing: All matrices are **shared across all time steps**, allowing the model to learn patterns independent of sequence position.

12.3 **Dimensions for Matrices**

Let d_{in} = input size, d_h = hidden size, and d_{out} = output size:

• **Input (x_t):** $\in \mathbb{R}^{d_{in}}$.

• **Hidden State (h_t):** $\in \mathbb{R}^{d_h}$.

• **Output (y_t):** $\in \mathbb{R}^{d_{out}}$.

• W : $d_h \times d_{in}$ | U : $d_h \times d_h$ | V : $d_{out} \times d_h$.

12.4 **Forward Pass Calculations**

The model processes elements one at a time:

1. **New State (h_t):** $h_t = \tanh(Uh_{t-1} + Wx_t + b_1)$

2. **Output (y_t):** $y_t = \text{softmax}(Vh_t + b_2)$ Output provides a probability distribution over classes.

12.5 **Backpropagation Through Time (BPTT)**

• **Dependency:** h_t influences current output y_t and subsequent state h_{t+1} .

• **Loss:** Total loss is the **sum of losses** at every time step.

• **Practice:** Often implemented as **truncated BPTT** using smaller sequence chunks.

12.6 **PyTorch Implementation**

```
import torch.nn as nn
# (input_size, hidden_size, num_layers)
rnn = nn.RNN(20, 10, 1)

# inp: (seq_len, batch, input_size)
# h0: (num_layers, batch, hidden_size)
output, hn = rnn(inp, h0)
```

PyTorch Output Details:

• **output:** Contains all hidden states (h_t) for the sequence.

• **hn:** Hidden state for only the **last time step**.

12.7 **Structure & Context**

Many-to-One: Sentiment (Compressed sequence)

Many-to-Many: POS Tagging (Local + Previous)

Bidirectional: Translation (Left + Right context)

Stacked: Complex NLP (Higher abstraction)

12.8 **Parameter Calculation**

Layer	Formula
RNN	$(d_{in} \times d_h) + d_h + (d_h \times d_h) + d_h$
Linear	$(d_h \times d_{out}) + d_{out}$

Note: Formulas include weights and biases for W, U, V .